

Chapter 21. Storage

Table of Contents

- 21.1. Synopsis
- 21.2. Adding Disks
- 21.3. Resizing and Growing Disks
- 21.4. USB Storage Devices
- 21.5. Creating and Using CD Media
- 21.6. Creating and Using DVD Media
- 21.7. Creating and Using Floppy Disks
- 21.8. Backup Basics
- 21.9. Memory Disks
- 21.10. File System Snapshots
- 21.11. Disk Quotas
- 21.12. Encrypting Disk Partitions
- 21.13. Encrypting Swap
- 21.14. Highly Available Storage (HAST)

21.1. Synopsis

This chapter covers the use of disks and storage media in FreeBSD. This includes SCSI and IDE disks, CD and DVD media, memory-backed disks, and USB storage devices.

Read this chapter to learn:

- How to add additional hard disks to a FreeBSD system.
- How to grow the size of a disk's partition on FreeBSD.
- How to configure FreeBSD to use USB storage devices.
- How to use CD and DVD media on a FreeBSD system.
- How to use the backup programs available under FreeBSD.
- How to set up memory disks.



- What file system snapshots are and how to use them efficiently.
- How to use quotas to limit disk space usage.
- How to encrypt disks and swap to secure them against attackers.
- How to configure a highly available storage network.

Before reading this chapter:

- Know how to [configure and install a new FreeBSD kernel](#).

21.2. Adding Disks

This section describes how to add a new SATA disk to a machine that currently only has a single drive. First, turn off the computer and install the drive in the computer following the instructions of the computer, controller, and drive manufacturers. Reboot the system and become `root`.

Inspect `/var/run/dmesg.boot` to ensure the new disk was found. In this example, the newly added SATA drive will appear as `ada1`.

For this example, a single large partition will be created on the new disk. The [GPT](#) partitioning scheme will be used in preference to the older and less versatile MBR scheme.

Note

If the disk to be added is not blank, old partition information can be removed with `gpart delete`. See [gpart\(8\)](#) for details.

The partition scheme is created, and then a single partition is added. To improve performance on newer disks with larger hardware block sizes, the partition is aligned to one megabyte boundaries:

```
# gpart create -s GPT ada1
# gpart add -t freebsd-ufs -a 1M ada1
```

Depending on use, several smaller partitions may be desired. See [gpart\(8\)](#) for options to create partitions smaller than a whole disk.

The disk partition information can be viewed with `gpart show`:

```
% gpart show ada1
=>      34  1465146988  ada1  GPT  (699G)
        34          2014      - free -  (1.0M)
      2048  1465143296      1  freebsd-ufs  (699G)
  1465145344      1678      - free -  (839K)
```

A file system is created in the new partition on the new disk:

```
# newfs -U /dev/ada1p1
```

An empty directory is created as a *mountpoint*, a location for mounting the new disk in the original disk's file system:

```
# mkdir /newdisk
```

Finally, an entry is added to **/etc/fstab** so the new disk will be mounted automatically at startup:

```
/dev/ada1p1    /newdisk      ufs      rw      2        2
```

The new disk can be mounted manually, without restarting the system:

```
# mount /newdisk
```

21.3. Resizing and Growing Disks

A disk's capacity can increase without any changes to the data already present. This happens commonly with virtual machines, when the virtual disk turns out to be too small and is enlarged. Sometimes a disk image is written to a USB memory stick, but does not use the full capacity. Here we describe how to resize or *grow* disk contents to take advantage of increased capacity.

Determine the device name of the disk to be resized by inspecting **/var/run/dmesg.boot**. In this example, there is only one SATA disk in the system, so the drive will appear as **ada0**.

List the partitions on the disk to see the current configuration:

```
# gpart show ada0
=>      34  83886013  ada0  GPT  (48G) [CORRUPT]
        34         128      1  freebsd-boot  (64k)
       162  79691648      2  freebsd-ufs   (38G)
    79691810  4194236      3  freebsd-swap  (2G)
    83886046         1      -   free -   (512B)
```

i Note

If the disk was formatted with the [GPT](#) partitioning scheme, it may show as "corrupted" because the GPT backup partition table is no longer at the end of the drive. Fix the backup partition table with `gpart`:





```
# gpart recover ada0
ada0 recovered
```

Now the additional space on the disk is available for use by a new partition, or an existing partition can be expanded:



```
# gpart show ada0
=>      34  102399933  ada0  GPT  (48G)
        34         128      1  freebsd-boot  (64k)
       162    79691648      2  freebsd-ufs   (38G)
    79691810    4194236      3  freebsd-swap (2G)
    83886046    18513921      - free -   (8.8G)
```

Partitions can only be resized into contiguous free space. Here, the last partition on the disk is the swap partition, but the second partition is the one that needs to be resized. Swap partitions only contain temporary data, so it can safely be unmounted, deleted, and then recreate the third partition after resizing the second partition.

Disable the swap partition:



```
# swapoff /dev/ada0p3
```

Delete the third partition, specified by the `-i` flag, from the disk *ada0*.



```
# gpart delete -i 3 ada0
ada0p3 deleted
# gpart show ada0
=>      34  102399933  ada0  GPT  (48G)
        34         128      1  freebsd-boot  (64k)
       162    79691648      2  freebsd-ufs   (38G)
    79691810    22708157      - free -   (10G)
```

Warning

There is risk of data loss when modifying the partition table of a mounted file system. It is best to perform the following steps on an unmounted file system while running from a live CD-ROM or USB device. However, if absolutely necessary, a mounted file system can be resized after disabling GEOM safety features:





```
# sysctl kern.geom.debugflags=16
```

Resize the partition, leaving room to recreate a swap partition of the desired size. The partition to resize is specified with `-i`, and the new desired size with `-s`. Optionally, alignment of the partition is controlled with `-a`. This only modifies the size of the partition. The file system in the partition will be expanded in a separate step.



```
# gpart resize -i 2 -s 47G -a 4k ada0
ada0p2 resized
# gpart show ada0
=>      34  102399933  ada0  GPT  (48G)
      34         128    1  freebsd-boot  (64k)
     162   98566144    2  freebsd-ufs  (47G)
  98566306   3833661    - free -  (1.8G)
```

Recreate the swap partition and activate it. If no size is specified with `-s`, all remaining space is used:



```
# gpart add -t freebsd-swap -a 4k ada0
ada0p3 added
# gpart show ada0
=>      34  102399933  ada0  GPT  (48G)
      34         128    1  freebsd-boot  (64k)
     162   98566144    2  freebsd-ufs  (47G)
  98566306   3833661    3  freebsd-swap  (1.8G)
# swapon /dev/ada0p3
```

Grow the UFS file system to use the new capacity of the resized partition:



```
# growfs /dev/ada0p2
Device is mounted read-write; resizing will result in temporary write suspension for /.
It's strongly recommended to make a backup before growing the file system.
OK to grow file system on /dev/ada0p2, mounted on /, from 38GB to 47GB? [Yes/No] Yes
super-block backups (for fsck -b #) at:
 80781312, 82063552, 83345792, 84628032, 85910272, 87192512, 88474752,
 89756992, 91039232, 92321472, 93603712, 94885952, 96168192, 97450432
```

If the file system is ZFS, the resize is triggered by running the `online` subcommand with `-e`:



Both the partition and the file system on it have now been resized to use the newly-available disk space.

21.4. USB Storage Devices

Many external storage solutions, such as hard drives, USB thumbdrives, and CD and DVD burners, use the Universal Serial Bus (USB). FreeBSD provides support for USB 1.x, 2.0, and 3.0 devices.

Note

USB 3.0 support is not compatible with some hardware, including Haswell (Lynx point) chipsets. If FreeBSD boots with a `failed with error 19` message, disable xHCI/USB3 in the system BIOS.

Support for USB storage devices is built into the **GENERIC** kernel. For a custom kernel, be sure that the following lines are present in the kernel configuration file:

```
device scbus      # SCSI bus (required for ATA/SCSI)
device da         # Direct Access (disks)
device pass       # Passthrough device (direct ATA/SCSI access)
device uhci       # provides USB 1.x support
device ohci       # provides USB 1.x support
device ehci       # provides USB 2.0 support
device xhci       # provides USB 3.0 support
device usb        # USB Bus (required)
device umass      # Disks/Mass storage - Requires scbus and da
device cd         # needed for CD and DVD burners
```

FreeBSD uses the `umass(4)` driver which uses the SCSI subsystem to access USB storage devices. Since any USB device will be seen as a SCSI device by the system, if the USB device is a CD or DVD burner, do *not* include `device atapicam` in a custom kernel configuration file.

The rest of this section demonstrates how to verify that a USB storage device is recognized by FreeBSD and how to configure the device so that it can be used.

21.4.1. Device Configuration

To test the USB configuration, plug in the USB device. Use `dmesg` to confirm that the drive appears in the system message buffer. It should look something like this:





```
umass0: <STECH Simple Drive, class 0/0, rev 2.00/1.04, addr 3> on usb0
umass0: SCSI over Bulk-Only; quirks = 0x0100
umass0:4:0:-1: Attached to scbus4
da0 at umass-sim0 bus 0 scbus4 target 0 lun 0
da0: <STECH Simple Drive 1.04> Fixed Direct Access SCSI-4 device
da0: Serial Number WD-WXE508CAN263
da0: 40.000MB/s transfers
da0: 152627MB (312581808 512 byte sectors: 255H 63S/T 19457C)
da0: quirks=0x2<NO_6_BYTE>
```

The brand, device node (**da0**), speed, and size will differ according to the device.

Since the USB device is seen as a SCSI one, `camcontrol` can be used to list the USB storage devices attached to the system:



```
# camcontrol devlist
<STECH Simple Drive 1.04>          at scbus4 target 0 lun 0 (pass3,da0)
```

Alternately, `usbconfig` can be used to list the device. Refer to [usbconfig\(8\)](#) for more information about this command.



```
# usbconfig
ugen0.3: <Simple Drive STECH> at usb0, cfg=0 md=HOST spd=HIGH (480Mbps) pwr=0N (2mA)
```

If the device has not been formatted, refer to [Adding Disks](#) for instructions on how to format and create partitions on the USB drive. If the drive comes with a file system, it can be mounted by `root` using the instructions in [“Mounting and Unmounting File Systems”](#).

Warning

Allowing untrusted users to mount arbitrary media, by enabling `vfs.usermount` as described below, should not be considered safe from a security point of view. Most file systems were not built to safeguard against malicious devices.

To make the device mountable as a normal user, one solution is to make all users of the device a member of the `operator` group using [pw\(8\)](#). Next, ensure that `operator` is able to read and write the device by adding these lines to **/etc/devfs.rules**:

```
[localrules=5]
add path 'da*' mode 0660 group operator
```



Note

If internal SCSI disks are also installed in the system, change the second line as follows:

```
add path 'da[3-9]*' mode 0660 group operator
```

This will exclude the first three SCSI disks (**da0** to **da2**) from belonging to the **operator** group. Replace 3 with the number of internal SCSI disks. Refer to [devfs.rules\(5\)](#) for more information about this file.

Next, enable the ruleset in **/etc/rc.conf**:

```
devfs_system_ruleset="localrules"
```

Then, instruct the system to allow regular users to mount file systems by adding the following line to **/etc/sysctl.conf**:

```
vfs.usermount=1
```

Since this only takes effect after the next reboot, use **sysctl** to set this variable now:

```
# sysctl vfs.usermount=1
vfs.usermount: 0 -> 1
```



The final step is to create a directory where the file system is to be mounted. This directory needs to be owned by the user that is to mount the file system. One way to do that is for **root** to create a subdirectory owned by that user as **/mnt/username**. In the following example, replace *username* with the login name of the user and *usergroup* with the user's primary group:

```
# mkdir /mnt/username
# chown username:usergroup /mnt/username
```



Suppose a USB thumbdrive is plugged in, and a device **/dev/da0s1** appears. If the device is formatted with a FAT file system, the user can mount it using:

```
% mount -t msdosfs -o -m=644, -M=755 /dev/da0s1 /mnt/username
```



Before the device can be unplugged, it *must* be unmounted first:




```
% umount /mnt/username
```

After device removal, the system message buffer will show messages similar to the following:

```
umass0: at uhub3, port 2, addr 3 (disconnected)
da0 at umass-sim0 bus 0 scbus4 target 0 lun 0
da0: <STECH Simple Drive 1.04> s/n WD-WXE508CAN263 detached
(da0:umass-sim0:0:0:0): Periph destroyed
```

21.4.2. Automounting Removable Media

USB devices can be automatically mounted by uncommenting this line in **/etc/auto_master**:

```
/media          -media          -nosuid
```

Then add these lines to **/etc/devd.conf**:

```
notify 100 {
    match "system" "GEOM";
    match "subsystem" "DEV";
    action "/usr/sbin/automount -c";
};
```

Reload the configuration if [autofs\(5\)](#) and [devd\(8\)](#) are already running:

```
# service automount restart
# service devd restart
```

[autofs\(5\)](#) can be set to start at boot by adding this line to **/etc/rc.conf**:

```
autofs_enable="YES"
```

[autofs\(5\)](#) requires [devd\(8\)](#) to be enabled, as it is by default.

Start the services immediately with:

```
# service automount start
# service automountd start
```

```
# service autounmountd start
# service devd start
```

Each file system that can be automatically mounted appears as a directory in **/media/**. The directory is named after the file system label. If the label is missing, the directory is named after the device node.

The file system is transparently mounted on the first access, and unmounted after a period of inactivity. Automounted drives can also be unmounted manually:

```
# automount -fu
```



This mechanism is typically used for memory cards and USB memory sticks. It can be used with any block device, including optical drives or iSCSILUNs.

21.5. Creating and Using CD Media

Compact Disc (CD) media provide a number of features that differentiate them from conventional disks. They are designed so that they can be read continuously without delays to move the head between tracks. While CD media do have tracks, these refer to a section of data to be read continuously, and not a physical property of the disk. The ISO 9660 file system was designed to deal with these differences.

The FreeBSD Ports Collection provides several utilities for burning and duplicating audio and data CDs. This chapter demonstrates the use of several command line utilities. For CD burning software with a graphical utility, consider installing the [sysutils/xcdroast](#) or [sysutils/k3b](#) packages or ports.

21.5.1. Supported Devices

The **GENERIC** kernel provides support for SCSI, USB, and ATAPICD readers and burners. If a custom kernel is used, the options that need to be present in the kernel configuration file vary by the type of device.

For a SCSI burner, make sure these options are present:

```
device scbus    # SCSI bus (required for ATA/SCSI)
device da       # Direct Access (disks)
device pass     # Passthrough device (direct ATA/SCSI access)
device cd       # needed for CD and DVD burners
```

For a USB burner, make sure these options are present:

```
device scbus    # SCSI bus (required for ATA/SCSI)
device da       # Direct Access (disks)
```



```
device pass      # Passthrough device (direct ATA/SCSI access)
device cd        # needed for CD and DVD burners
device uhci      # provides USB 1.x support
device ohci      # provides USB 1.x support
device ehci      # provides USB 2.0 support
device xhci      # provides USB 3.0 support
device usb       # USB Bus (required)
device umass     # Disks/Mass storage - Requires scbus and da
```

For an ATAPI burner, make sure these options are present:

```
device ata       # Legacy ATA/SATA controllers
device scbus     # SCSI bus (required for ATA/SCSI)
device pass      # Passthrough device (direct ATA/SCSI access)
device cd        # needed for CD and DVD burners
```

i Note

On FreeBSD versions prior to 10.x, this line is also needed in the kernel configuration file if the burner is an ATAPI device:

```
device atapicam
```

Alternately, this driver can be loaded at boot time by adding the following line to **/boot/loader.conf**:

```
atapicam_load="YES"
```

This will require a reboot of the system as this driver can only be loaded at boot time.

To verify that FreeBSD recognizes the device, run `dmesg` and look for an entry for the device. On systems prior to 10.x, the device name in the first line of the output will be **acd0** instead of **cd0**.

```
% dmesg | grep cd
cd0 at ahcich1 bus 0 scbus1 target 0 lun 0
cd0: <HL-DT-ST DVDROM GU70N LT20> Removable CD-ROM SCSI-0 device
cd0: Serial Number M30D3S34152
cd0: 150.000MB/s transfers (SATA 1.x, UDMA6, ATAPI 12bytes, PIO 8192bytes)
cd0: Attempt to query device size failed: NOT READY, Medium not present - tray closed
```



21.5.2. Burning a CD

In FreeBSD, `cdrecord` can be used to burn CDs. This command is installed with the `sysutils/cdrtools` package or port.

While `cdrecord` has many options, basic usage is simple. Specify the name of the ISO file to burn and, if the system has multiple burner devices, specify the name of the device to use:

```
# cdrecord dev=device imagefile.iso
```

To determine the device name of the burner, use `-scanbus` which might produce results like this:

```
# cdrecord -scanbus
ProDVD-ProBD-Clone 3.00 (amd64-unknown-freebsd10.0) Copyright (C) 1995-2010 Jörg Schilling
Using libscg version 'schily-0.9'
scsibus0:
    0,0,0    0) 'SEAGATE ' 'ST39236LW      ' '0004' Disk
    0,1,0    1) 'SEAGATE ' 'ST39173W      ' '5958' Disk
    0,2,0    2) *
    0,3,0    3) 'iomega  ' 'jaz 1GB        ' 'J.86' Removable Disk
    0,4,0    4) 'NEC      ' 'CD-ROM DRIVE:466' '1.26' Removable CD-ROM
    0,5,0    5) *
    0,6,0    6) *
    0,7,0    7) *
scsibus1:
    1,0,0   100) *
    1,1,0   101) *
    1,2,0   102) *
    1,3,0   103) *
    1,4,0   104) *
    1,5,0   105) 'YAMAHA  ' 'CRW4260      ' '1.0q' Removable CD-ROM
    1,6,0   106) 'ARTEC   ' 'AM12S         ' '1.06' Scanner
    1,7,0   107) *
```

Locate the entry for the CD burner and use the three numbers separated by commas as the value for `dev`. In this case, the Yamaha burner device is `1,5,0`, so the appropriate input to specify that device is `dev=1,5,0`. Refer to the manual page for `cdrecord` for other ways to specify this value and for information on writing audio tracks and controlling the write speed.

Alternately, run the following command to get the device address of the burner:

```
# camcontrol devlist
<MATSHITA CDRW/DVD UJDA740 1.00>   at scbus1 target 0 lun 0 (cd0,pass0)
```

Use the numeric values for `scbus`, `target`, and `lun`. For this example, `1,0,0` is the device name to use.

21.5.3. Writing Data to an ISO File System

In order to produce a data CD, the data files that are going to make up the tracks on the CD must be prepared before they can be burned to the CD. In FreeBSD, [sysutils/cdrtools](#) installs `mkisofs`, which can be used to produce an ISO 9660 file system that is an image of a directory tree within a UNIX® file system. The simplest usage is to specify the name of the ISO file to create and the path to the files to place into the ISO 9660 file system:

```
# mkisofs -o imagefile.iso /path/to/tree
```

This command maps the file names in the specified path to names that fit the limitations of the standard ISO 9660 file system, and will exclude files that do not meet the standard for ISO file systems.

A number of options are available to overcome the restrictions imposed by the standard. In particular, `-R` enables the Rock Ridge extensions common to UNIX® systems and `-J` enables Joliet extensions used by Microsoft® systems.

For CDs that are going to be used only on FreeBSD systems, `-U` can be used to disable all filename restrictions. When used with `-R`, it produces a file system image that is identical to the specified FreeBSD tree, even if it violates the ISO 9660 standard.

The last option of general use is `-b`. This is used to specify the location of a boot image for use in producing an "El Torito" bootable CD. This option takes an argument which is the path to a boot image from the top of the tree being written to the CD. By default, `mkisofs` creates an ISO image in "floppy disk emulation" mode, and thus expects the boot image to be exactly 1200, 1440 or 2880 KB in size. Some boot loaders, like the one used by the FreeBSD distribution media, do not use emulation mode. In this case, `-no-emul-boot` should be used. So, if `/tmp/myboot` holds a bootable FreeBSD system with the boot image in `/tmp/myboot/boot/cdboot`, this command would produce `/tmp/bootable.iso`:

```
# mkisofs -R -no-emul-boot -b boot/cdboot -o /tmp/bootable.iso /tmp/myboot
```

The resulting ISO image can be mounted as a memory disk with:

```
# mdconfig -a -t vnode -f /tmp/bootable.iso -u 0
# mount -t cd9660 /dev/md0 /mnt
```

One can then verify that `/mnt` and `/tmp/myboot` are identical.



There are many other options available for `mkisofs` to fine-tune its behavior. Refer to [mkisofs\(8\)](#) for details.

i Note

It is possible to copy a data CD to an image file that is functionally equivalent to the image file created with `mkisofs`. To do so, use `dd` with the device name as the input file and the name of the ISO to create as the output file:

```
# dd if=/dev/cd0 of=file.iso bs=2048
```



The resulting image file can be burned to CD as described in [Burning a CD](#).

21.5.4. Using Data CDs

Once an ISO has been burned to a CD, it can be mounted by specifying the file system type, the name of the device containing the CD, and an existing mount point:

```
# mount -t cd9660 /dev/cd0 /mnt
```



Since `mount` assumes that a file system is of type `ufs`, an `Incorrect super block` error will occur if `-t cd9660` is not included when mounting a data CD.

While any data CD can be mounted this way, disks with certain ISO 9660 extensions might behave oddly. For example, Joliet disks store all filenames in two-byte Unicode characters. If some non-English characters show up as question marks, specify the local charset with `-C`. For more information, refer to [mount_cd9660\(8\)](#).

i Note

In order to do this character conversion with the help of `-C`, the kernel requires the `cd9660_iconv.ko` module to be loaded. This can be done either by adding this line to `loader.conf`:

```
cd9660_iconv_load="YES"
```

and then rebooting the machine, or by directly loading the module with `kldload`.



Occasionally, `Device not configured` will be displayed when trying to mount a data CD. This usually means that the CD drive has not detected a disk in the tray, or that the drive is not visible on the bus. It can take a couple of seconds for a CD drive to detect media, so be patient.

Sometimes, a SCSI CD drive may be missed because it did not have enough time to answer the bus reset. To resolve this, a custom kernel can be created which increases the default SCSI delay. Add the following option to the custom kernel configuration file and rebuild the kernel using the instructions in [“Building and Installing a Custom Kernel”](#):

```
options SCSI_DELAY=15000
```

This tells the SCSI bus to pause 15 seconds during boot, to give the CD drive every possible chance to answer the bus reset.

Note

It is possible to burn a file directly to CD, without creating an ISO 9660 file system. This is known as burning a raw data CD and some people do this for backup purposes.

This type of disk can not be mounted as a normal data CD. In order to retrieve the data burned to such a CD, the data must be read from the raw device node. For example, this command will extract a compressed tar file located on the second CD device into the current working directory:

```
# tar xzvf /dev/cd1
```

In order to mount a data CD, the data must be written using `mkisofs`.

21.5.5. Duplicating Audio CDs

To duplicate an audio CD, extract the audio data from the CD to a series of files, then write these files to a blank CD.

[Duplicating an Audio CD](#) describes how to duplicate and burn an audio CD. If the FreeBSD version is less than 10.0 and the device is ATAPI, the `atapicam` module must be first loaded using the instructions in [Supported Devices](#).

Procedure: Duplicating an Audio CD

1. The [sysutils/cdrtools](#) package or port installs `cdda2wav`. This command can be used to extract all of the audio tracks, with each track written to a separate WAV file in the current working directory:



```
% cdda2wav -vall -B -0wav
```

A device name does not need to be specified if there is only one CD device on the system. Refer to the `cdda2wav` manual page for instructions on how to specify a device and to learn more about the other options available for this command.

2. Use `cdrecord` to write the `.wav` files:

```
% cdrecord -v dev=2,0 -dao -useinfo *.wav
```

Make sure that `2,0` is set appropriately, as described in [Burning a CD](#).

21.6. Creating and Using DVD Media

Compared to the CD, the DVD is the next generation of optical media storage technology. The DVD can hold more data than any CD and is the standard for video publishing.

Five physical recordable formats can be defined for a recordable DVD:

- DVD-R: This was the first DVD recordable format available. The DVD-R standard is defined by the [DVD Forum](#). This format is write once.
- DVD-RW: This is the rewritable version of the DVD-R standard. A DVD-RW can be rewritten about 1000 times.
- DVD-RAM: This is a rewritable format which can be seen as a removable hard drive. However, this media is not compatible with most DVD-ROM drives and DVD-Video players as only a few DVD writers support the DVD-RAM format. Refer to [Using a DVD-RAM](#) for more information on DVD-RAM use.
- DVD+RW: This is a rewritable format defined by the [DVD+RW Alliance](#). A DVD+RW can be rewritten about 1000 times.
- DVD+R: This format is the write once variation of the DVD+RW format.

A single layer recordable DVD can hold up to 4,700,000,000 bytes which is actually 4.38 GB or 4485 MB as 1 kilobyte is 1024 bytes.



i Note

A distinction must be made between the physical media and the application. For example, a DVD-Video is a specific file layout that can be written on any recordable DVD physical media such as DVD-R, DVD+R, or DVD-RW. Before choosing the type of media, ensure that both the burner and the DVD-Video player are compatible with the media under consideration.

21.6.1. Configuration

To perform DVD recording, use [growisofs\(1\)](#). This command is part of the [sysutils/dvd+rw-tools](#) utilities which support all DVD media types.

These tools use the SCSI subsystem to access the devices, therefore [ATAPI/CAM support](#) must be loaded or statically compiled into the kernel. This support is not needed if the burner uses the USB interface. Refer to [USB Storage Devices](#) for more details on USB device configuration.

DMA access must also be enabled for ATAPI devices, by adding the following line to **/boot/loader.conf**:

```
hw.ata.atapi_dma="1"
```

Before attempting to use dvd+rw-tools, consult the [Hardware Compatibility Notes](#).

i Note

For a graphical user interface, consider using [sysutils/k3b](#) which provides a user friendly interface to [growisofs\(1\)](#) and many other burning tools.

21.6.2. Burning Data DVDs

Since [growisofs\(1\)](#) is a front-end to [mkisofs](#), it will invoke [mkisofs\(8\)](#) to create the file system layout and perform the write on the DVD. This means that an image of the data does not need to be created before the burning process.

To burn to a DVD+R or a DVD-R the data in **/path/to/data**, use the following command:

```
# growisofs -dvd-compat -Z /dev/cd0 -J -R /path/to/data
```



In this example, **-J -R** is passed to [mkisofs\(8\)](#) to create an ISO 9660 file system with Joliet and Rock Ridge extensions. Refer to [mkisofs\(8\)](#) for more details.



For the initial session recording, `-Z` is used for both single and multiple sessions. Replace `/dev/cd0`, with the name of the DVD device. Using `-dvd-compat` indicates that the disk will be closed and that the recording will be unappendable. This should also provide better media compatibility with DVD-ROM drives.

To burn a pre-mastered image, such as *imagefile.iso*, use:

```
# growisofs -dvd-compat -Z /dev/cd0=imagefile.iso
```

The write speed should be detected and automatically set according to the media and the drive being used. To force the write speed, use `-speed=`. Refer to [growisofs\(1\)](#) for example usage.

i Note

In order to support working files larger than 4.38GB, an UDF/ISO-9660 hybrid file system must be created by passing `-udf -iso-level 3` to [mkisofs\(8\)](#) and all related programs, such as [growisofs\(1\)](#). This is required only when creating an ISO image file or when writing files directly to a disk. Since a disk created this way must be mounted as an UDF file system with [mount_udf\(8\)](#), it will be usable only on an UDF aware operating system. Otherwise it will look as if it contains corrupted files.

To create this type of ISO file:

```
% mkisofs -R -J -udf -iso-level 3 -o imagefile.iso /path/to/data
```

To burn files directly to a disk:

```
# growisofs -dvd-compat -udf -iso-level 3 -Z /dev/cd0 -J -R /path/to/data
```

When an ISO image already contains large files, no additional options are required for [growisofs\(1\)](#) to burn that image on a disk.

Be sure to use an up-to-date version of [sysutils/cdrtools](#), which contains [mkisofs\(8\)](#), as an older version may not contain large files support. If the latest version does not work, install [sysutils/cdrtools-devel](#) and read its [mkisofs\(8\)](#).

21.6.3. Burning a DVD-Video



A DVD-Video is a specific file layout based on the ISO 9660 and micro-UDF (M-UDF) specifications. Since DVD-Video presents a specific data structure hierarchy, a particular program such as [multimedia/dvdauthor](#) is needed to author the DVD.

If an image of the DVD-Video file system already exists, it can be burned in the same way as any other image. If `dvdauthor` was used to make the DVD and the result is in `/path/to/video`, the following command should be used to burn the DVD-Video:

```
# growisofs -Z /dev/cd0 -dvd-video /path/to/video
```

`-dvd-video` is passed to [mkisofs\(8\)](#) to instruct it to create a DVD-Video file system layout. This option implies the `-dvd-compat` [growisofs\(1\)](#) option.

21.6.4. Using a DVD+RW

Unlike CD-RW, a virgin DVD+RW needs to be formatted before first use. It is *recommended* to let [growisofs\(1\)](#) take care of this automatically whenever appropriate. However, it is possible to use `dvd+rw-format` to format the DVD+RW:

```
# dvd+rw-format /dev/cd0
```

Only perform this operation once and keep in mind that only virgin DVD+RW medias need to be formatted. Once formatted, the DVD+RW can be burned as usual.

To burn a totally new file system and not just append some data onto a DVD+RW, the media does not need to be blanked first. Instead, write over the previous recording like this:

```
# growisofs -Z /dev/cd0 -J -R /path/to/newdata
```

The DVD+RW format supports appending data to a previous recording. This operation consists of merging a new session to the existing one as it is not considered to be multi-session writing. [growisofs\(1\)](#) will *grow* the ISO 9660 file system present on the media.

For example, to append data to a DVD+RW, use the following:

```
# growisofs -M /dev/cd0 -J -R /path/to/nextdata
```

The same [mkisofs\(8\)](#) options used to burn the initial session should be used during next writes.



Note

Use `-dvd-compat` for better media compatibility with DVD-ROM drives. When using DVD+RW, this option will not prevent the addition of data.

To blank the media, use:

```
# growisofs -Z /dev/cd0=/dev/zero
```



21.6.5. Using a DVD-RW

A DVD-RW accepts two disc formats: incremental sequential and restricted overwrite. By default, DVD-RW discs are in sequential format.

A virgin DVD-RW can be directly written without being formatted. However, a non-virgin DVD-RW in sequential format needs to be blanked before writing a new initial session.

To blank a DVD-RW in sequential mode:

```
# dvd+rw-format -blank=full /dev/cd0
```



Note

A full blanking using `-blank=full` will take about one hour on a 1x media. A fast blanking can be performed using `-blank`, if the DVD-RW will be recorded in Disk-At-Once (DAO) mode. To burn the DVD-RW in DAO mode, use the command:

```
# growisofs -use-the-force-luke=dao -Z /dev/cd0=imagefile.iso
```



Since [growisofs\(1\)](#) automatically attempts to detect fast blanked media and engage DAO write, `-use-the-force-luke=dao` should not be required.

One should instead use restricted overwrite mode with any DVD-RW as this format is more flexible than the default of incremental sequential.

To write data on a sequential DVD-RW, use the same instructions as for the other DVD formats:





```
# growisofs -Z /dev/cd0 -J -R /path/to/data
```

To append some data to a previous recording, use `-M` with [growisofs\(1\)](#). However, if data is appended on a DVD-RW in incremental sequential mode, a new session will be created on the disc and the result will be a multi-session disc.

A DVD-RW in restricted overwrite format does not need to be blanked before a new initial session. Instead, overwrite the disc with `-Z`. It is also possible to grow an existing ISO 9660 file system written on the disc with `-M`. The result will be a one-session DVD.

To put a DVD-RW in restricted overwrite format, the following command must be used:



```
# dvd+rw-format /dev/cd0
```

To change back to sequential format, use:



```
# dvd+rw-format -blank=full /dev/cd0
```

21.6.6. Multi-Session

Few DVD-ROM drives support multi-session DVDs and most of the time only read the first session. DVD+R, DVD-R and DVD-RW in sequential format can accept multiple sessions. The notion of multiple sessions does not exist for the DVD+RW and the DVD-RW restricted overwrite formats.

Using the following command after an initial non-closed session on a DVD+R, DVD-R, or DVD-RW in sequential format, will add a new session to the disc:



```
# growisofs -M /dev/cd0 -J -R /path/to/nextdata
```

Using this command with a DVD+RW or a DVD-RW in restricted overwrite mode will append data while merging the new session to the existing one. The result will be a single-session disc. Use this method to add data after an initial write on these types of media.

Note

Since some space on the media is used between each session to mark the end and start of sessions, one should add sessions with a large amount of data to optimize media space. The number of sessions is limited to 154 for a DVD+R, about 2000 for a DVD-R, and 127 for a DVD+R Double Layer.



21.6.7. For More Information

To obtain more information about a DVD, use `dvd+rw-mediainfo /dev/cd0` while the disc is in the specified drive.

More information about dvd+rw-tools can be found in [growisofs\(1\)](#), on the [dvd+rw-tools web site](#), and in the [cdwrite mailing list](#) archives.

i Note

When creating a problem report related to the use of dvd+rw-tools, always include the output of `dvd+rw-mediainfo`.

21.6.8. Using a DVD-RAM

DVD-RAM writers can use either a SCSI or ATAPI interface. For ATAPI devices, DMA access has to be enabled by adding the following line to **/boot/loader.conf**:

```
hw.ata.atapi_dma="1"
```

A DVD-RAM can be seen as a removable hard drive. Like any other hard drive, the DVD-RAM must be formatted before it can be used. In this example, the whole disk space will be formatted with a standard UFS2 file system:

```
# dd if=/dev/zero of=/dev/acd0 bs=2k count=1
# bsdlabel -Bw acd0
# newfs /dev/acd0
```

The DVD device, **acd0**, must be changed according to the configuration.

Once the DVD-RAM has been formatted, it can be mounted as a normal hard drive:

```
# mount /dev/acd0 /mnt
```

Once mounted, the DVD-RAM will be both readable and writeable.

21.7. Creating and Using Floppy Disks

This section explains how to format a 3.5 inch floppy disk in FreeBSD.



Procedure: Steps to Format a Floppy

A floppy disk needs to be low-level formatted before it can be used. This is usually done by the vendor, but formatting is a good way to check media integrity. To low-level format the floppy disk on FreeBSD, use [fdformat\(8\)](#). When using this utility, make note of any error messages, as these can help determine if the disk is good or bad.

1. To format the floppy, insert a new 3.5 inch floppy disk into the first floppy drive and issue:

```
# /usr/sbin/fdformat -f 1440 /dev/fd0
```

2. After low-level formatting the disk, create a disk label as it is needed by the system to determine the size of the disk and its geometry. The supported geometry values are listed in [/etc/disktab](#).

To write the disk label, use [bsdlabel\(8\)](#):

```
# /sbin/bsdlabel -B -w /dev/fd0 fd1440
```

3. The floppy is now ready to be high-level formatted with a file system. The floppy's file system can be either UFS or FAT, where FAT is generally a better choice for floppies.

To format the floppy with FAT, issue:

```
# /sbin/newfs_msdos /dev/fd0
```

The disk is now ready for use. To use the floppy, mount it with [mount_msdosfs\(8\)](#). One can also install and use [emulators/mtools](#) from the Ports Collection.

21.8. Backup Basics

Implementing a backup plan is essential in order to have the ability to recover from disk failure, accidental file deletion, random file corruption, or complete machine destruction, including destruction of on-site backups.

The backup type and schedule will vary, depending upon the importance of the data, the granularity needed for file restores, and the amount of acceptable downtime. Some possible backup techniques include:



- Archives of the whole system, backed up onto permanent, off-site media. This provides protection against all of the problems listed above, but is slow and inconvenient to restore from, especially for non-privileged users.
- File system snapshots, which are useful for restoring deleted files or previous versions of files.
- Copies of whole file systems or disks which are synchronized with another system on the network using a scheduled [net/rsync](#).
- Hardware or software RAID, which minimizes or avoids downtime when a disk fails.

Typically, a mix of backup techniques is used. For example, one could create a schedule to automate a weekly, full system backup that is stored off-site and to supplement this backup with hourly ZFS snapshots. In addition, one could make a manual backup of individual directories or files before making file edits or deletions.

This section describes some of the utilities which can be used to create and manage backups on a FreeBSD system.

21.8.1. File System Backups

The traditional UNIX® programs for backing up a file system are [dump\(8\)](#), which creates the backup, and [restore\(8\)](#), which restores the backup. These utilities work at the disk block level, below the abstractions of the files, links, and directories that are created by file systems. Unlike other backup software, `dump` backs up an entire file system and is unable to backup only part of a file system or a directory tree that spans multiple file systems. Instead of writing files and directories, `dump` writes the raw data blocks that comprise files and directories.

Note

If `dump` is used on the root directory, it will not back up `/home`, `/usr`, or many other directories since these are typically mount points for other file systems or symbolic links into those file systems.

When used to restore data, `restore` stores temporary files in `/tmp/` by default. When using a recovery disk with a small `/tmp`, set `TMPDIR` to a directory with more free space for the restore to succeed.

When using `dump`, be aware that some quirks remain from its early days in Version 6 of AT&T UNIX®, circa 1975. The default parameters assume a backup to a 9-track tape, rather than to another type of media or to the high-density tapes available today. These defaults must be overridden on the command line.

It is possible to backup a file system across the network to another system or a tape drive attached to another computer. While the [rdump\(8\)](#) and [rrestore\(8\)](#) utilities can be used for this purpose, they are not considered to be secure.



Instead, one can use `dump` and `restore` more securely over an SSH connection. This example creates a full, compressed backup of `/usr` and sends the backup file to the specified host over an SSH connection.

Example 1. Using `dump` over ssh



```
# /sbin/dump -0uan -f - /usr | gzip -2 | ssh -c blowfish \  
targetuser@targetmachine.example.com dd of=/mybigfiles/dump-usr-l0.gz
```

This example sets `RSH` in order to write the backup to a tape drive on a remote system over an SSH connection:

Example 2. Using `dump` over ssh with `RSH` Set



```
# env RSH=/usr/bin/ssh /sbin/dump -0uan -f targetuser@targetmachine.example.com:/dev/sa0 /usr
```

Tip

Systems using the [Z file system \(ZFS\)](#) can make use of [zfs\(8\)](#) for creating snapshots, as well as [sending and receiving](#) them to/from remote systems.

21.8.2. Directory Backups

Several built-in utilities are available for backing up and restoring specified files and directories as needed.

A good choice for making a backup of all of the files in a directory is [tar\(1\)](#). This utility dates back to Version 6 of AT&T UNIX® and by default assumes a recursive backup to a local tape device. Switches can be used to instead specify the name of a backup file.

This example creates a compressed backup of the current directory and saves it to `/tmp/mybackup.tgz`. When creating a backup file, make sure that the backup is not saved to the same directory that is being backed up.

Example 3. Backing Up the Current Directory with `tar`





```
# tar czvf /tmp/mybackup.tgz .
```

To restore the entire backup, `cd` into the directory to restore into and specify the name of the backup. Note that this will overwrite any newer versions of files in the restore directory. When in doubt, restore to a temporary directory or specify the name of the file within the backup to restore.

Example 4. Restoring Up the Current Directory with `tar`



```
# tar xzvf /tmp/mybackup.tgz
```

There are dozens of available switches which are described in [tar\(1\)](#). This utility also supports the use of exclude patterns to specify which files should not be included when backing up the specified directory or restoring files from a backup.

To create a backup using a specified list of files and directories, [cpio\(1\)](#) is a good choice. Unlike `tar`, `cpio` does not know how to walk the directory tree and it must be provided the list of files to backup.

For example, a list of files can be created using `ls` or `find`. This example creates a recursive listing of the current directory which is then piped to `cpio` in order to create an output backup file named **`/tmp/mybackup.cpio`**.

Example 5. Using `ls` and `cpio` to Make a Recursive Backup of the Current Directory



```
# ls -R | cpio -ovF /tmp/mybackup.cpio
```

A backup utility which tries to bridge the features provided by `tar` and `cpio` is [pax\(1\)](#). Over the years, the various versions of `tar` and `cpio` became slightly incompatible. POSIX® created `pax` which attempts to read and write many of the various `cpio` and `tar` formats, plus new formats of its own.

The `pax` equivalent to the previous examples would be:

Example 6. Backing Up the Current Directory with `pax`





```
# pax -wf /tmp/mybackup.pax .
```

21.8.3. Using Data Tapes for Backups

While tape technology has continued to evolve, modern backup systems tend to combine off-site backups with local removable media. FreeBSD supports any tape drive that uses SCSI, such as LTO or DAT. There is limited support for SATA and USB tape drives.

For SCSI tape devices, FreeBSD uses the [sa\(4\)](#) driver and the **/dev/sa0**, **/dev/nsa0**, and **/dev/esa0** devices. The physical device name is **/dev/sa0**. When **/dev/nsa0** is used, the backup application will not rewind the tape after writing a file, which allows writing more than one file to a tape. Using **/dev/esa0** ejects the tape after the device is closed.

In FreeBSD, **mt** is used to control operations of the tape drive, such as seeking through files on a tape or writing tape control marks to the tape. For example, the first three files on a tape can be preserved by skipping past them before writing a new file:



```
# mt -f /dev/nsa0 fsf 3
```

This utility supports many operations. Refer to [mt\(1\)](#) for details.

To write a single file to tape using **tar**, specify the name of the tape device and the file to backup:



```
# tar cvf /dev/sa0 file
```

To recover files from a **tar** archive on tape into the current directory:



```
# tar xvf /dev/sa0
```

To backup a UFS file system, use **dump**. This examples backs up **/usr** without rewinding the tape when finished:



```
# dump -0aL -b64 -f /dev/nsa0 /usr
```

To interactively restore files from a **dump** file on tape into the current directory:



21.8.4. Third-Party Backup Utilities

The FreeBSD Ports Collection provides many third-party utilities which can be used to schedule the creation of backups, simplify tape backup, and make backups easier and more convenient. Many of these applications are client/server based and can be used to automate the backups of a single system or all of the computers in a network.

Popular utilities include:

- Amanda ([misc/amanda-server](#) and [misc/amanda-client](#)),
- Bacula ([sysutils/bacula13-server](#) and [sysutils/bacula13-client](#)),
- Bareos ([sysutils/bareos-server](#) and [sysutils/bareos-client](#)),
- [net/rsync](#),
- [sysutils/duply](#), and
- [sysutils/duplicity](#).

21.8.5. Emergency Recovery

In addition to regular backups, it is recommended to perform the following steps as part of an emergency preparedness plan.

Create a print copy of the output of the following commands:

- `gpart show`
- `more /etc/fstab`
- `pkg prime-list`
- `dmesg`

Store this printout and a copy of the installation media in a secure location. Should an emergency restore be needed, boot into the installation media and select `Live CD` to access a rescue shell. This rescue mode can be used to view the current state of the system, and if needed, to reformat disks and restore data from backups.

Next, test the rescue shell and the backups. Make notes of the procedure. Store these notes with the media, the printouts, and the backups. These notes may prevent the inadvertent destruction of the backups while under the stress of performing an emergency recovery.



For an added measure of security, store the latest backup at a remote location which is physically separated from the computers and disk drives by a significant distance.

21.9. Memory Disks

In addition to physical disks, FreeBSD also supports the creation and use of memory disks. One possible use for a memory disk is to access the contents of an ISO file system without the overhead of first burning it to a CD or DVD, then mounting the CD/DVD media.

In FreeBSD, the [md\(4\)](#) driver is used to provide support for memory disks. The **GENERIC** kernel includes this driver. When using a custom kernel configuration file, ensure it includes this line:

```
device md
```

21.9.1. Attaching and Detaching Existing Images

To mount an existing file system image, use `mdconfig` to specify the name of the ISO file and a free unit number. Then, refer to that unit number to mount it on an existing mount point. Once mounted, the files in the ISO will appear in the mount point. This example attaches *diskimage.iso* to the memory device **/dev/md0** then mounts that memory device on **/mnt**:

```
# mdconfig -f diskimage.iso -u 0
# mount -t cd9660 /dev/md0 /mnt
```

Notice that `-t cd9660` was used to mount an ISO format. If a unit number is not specified with `-u`, `mdconfig` will automatically allocate an unused memory device and output the name of the allocated unit, such as **md4**. Refer to [mdconfig\(8\)](#) for more details about this command and its options.

When a memory disk is no longer in use, its resources should be released back to the system. First, unmount the file system, then use `mdconfig` to detach the disk from the system and release its resources. To continue this example:

```
# umount /mnt
# mdconfig -d -u 0
```

To determine if any memory disks are still attached to the system, type `mdconfig -l`.

21.9.2. Creating a File- or Memory-Backed Memory Disk

FreeBSD also supports memory disks where the storage to use is allocated from either a hard disk or an area of memory. The first method is commonly referred to as a file-backed file system and the second method as a memory-backed file system. Both types can be created using `mdconfig`.

To create a new memory-backed file system, specify a type of `swap` and the size of the memory disk to create. Then, format the memory disk with a file system and mount as usual. This example creates a 5M memory disk on unit `1`. That memory disk is then formatted with the UFS file system before it is mounted:

```
# mdconfig -a -t swap -s 5m -u 1
# newfs -U md1
/dev/md1: 5.0MB (10240 sectors) block size 16384, fragment size 2048
      using 4 cylinder groups of 1.27MB, 81 blks, 192 inodes.
      with soft updates
super-block backups (for fsck -b #) at:
 160, 2752, 5344, 7936
# mount /dev/md1 /mnt
# df /mnt
Filesystem 1K-blocks Used Avail Capacity Mounted on
/dev/md1      4718    4  4338      0%    /mnt
```

To create a new file-backed memory disk, first allocate an area of disk to use. This example creates an empty 5MB file named **newimage**:

```
# dd if=/dev/zero of=newimage bs=1k count=5k
5120+0 records in
5120+0 records out
```

Next, attach that file to a memory disk, label the memory disk and format it with the UFS file system, mount the memory disk, and verify the size of the file-backed disk:

```
# mdconfig -f newimage -u 0
# bsdlablel -w md0 auto
# newfs -U md0a
/dev/md0a: 5.0MB (10224 sectors) block size 16384, fragment size 2048
      using 4 cylinder groups of 1.25MB, 80 blks, 192 inodes.
super-block backups (for fsck -b #) at:
 160, 2720, 5280, 7840
# mount /dev/md0a /mnt
# df /mnt
Filesystem 1K-blocks Used Avail Capacity Mounted on
/dev/md0a    4710    4  4330      0%    /mnt
```

It takes several commands to create a file- or memory-backed file system using `mdconfig`. FreeBSD also comes with `mdmfs` which automatically configures a memory disk, formats it with the UFS file system, and mounts it. For example, after creating *newimage* with `dd`, this one command is equivalent to running the `bsdlablel`, `newfs`, and `mount` commands shown above.

```
# mdmfs -F newimage -s 5m md0 /mnt
```



To instead create a new memory-based memory disk with `mdmfs`, use this one command:

```
# mdmfs -s 5m md1 /mnt
```



If the unit number is not specified, `mdmfs` will automatically select an unused memory device. For more details about `mdmfs`, refer to [mdmfs\(8\)](#).

21.10. File System Snapshots

FreeBSD offers a feature in conjunction with soft updates: file system snapshots.

UFS snapshots allow a user to create images of specified file systems, and treat them as a file. When using the [Z file system \(ZFS\)](#), refer to [Managing Snapshots](#) on how to use snapshots.

Snapshot files must be created in the file system that the action is performed on, and a user may create no more than 20 snapshots per file system. Active snapshots are recorded in the superblock so they are persistent across unmount and remount operations along with system reboots. When a snapshot is no longer required, it can be removed using [rm\(1\)](#). While snapshots may be removed in any order, all the used space may not be acquired because another snapshot will possibly claim some of the released blocks.

The un-alterable `snapshot` file flag is set by [mksnap_ffs\(8\)](#) after initial creation of a snapshot file. [unlink\(1\)](#) makes an exception for snapshot files since it allows them to be removed.

Snapshots are created using [mount\(8\)](#). To place a snapshot of `/var` in the file `/var/snapshot/snap`, use the following command:

```
# mount -u -o snapshot /var/snapshot/snap /var
```



Alternatively, use [mksnap_ffs\(8\)](#) to create the snapshot:

```
# mksnap_ffs /var /var/snapshot/snap
```



One can find snapshot files on a file system, such as `/var`, using [find\(1\)](#):

```
# find /var -flags snapshot
```



Once a snapshot has been created, it has several uses:



- Some administrators will use a snapshot file for backup purposes, because the snapshot can be transferred to CDs or tape.
- The file system integrity checker, `fsck(8)`, may be run on the snapshot. Assuming that the file system was clean when it was mounted, this should always provide a clean and unchanging result.
- Running `dump(8)` on the snapshot will produce a dump file that is consistent with the file system and the timestamp of the snapshot. `dump(8)` can also take a snapshot, create a dump image, and then remove the snapshot in one command by using `-L`.
- The snapshot can be mounted as a frozen image of the file system. To `mount(8)` the snapshot `/var/snapshot/snap` run:

```
# mdconfig -a -t vnode -o readonly -f /var/snapshot/snap -u 4
# mount -r /dev/md4 /mnt
```

The frozen `/var` is now available through `/mnt`. Everything will initially be in the same state it was during the snapshot creation time. The only exception is that any earlier snapshots will appear as zero length files. To unmount the snapshot, use:

```
# umount /mnt
# mdconfig -d -u 4
```

For more information about `softupdates` and file system snapshots, including technical papers, visit Marshall Kirk McKusick's website at <http://www.mckusick.com/>.

21.11. Disk Quotas

Disk quotas can be used to limit the amount of disk space or the number of files a user or members of a group may allocate on a per-file system basis. This prevents one user or group of users from consuming all of the available disk space.

This section describes how to configure disk quotas for the UFS file system. To configure quotas on the ZFS file system, refer to [Dataset, User, and Group Quotas](#)

21.11.1. Enabling Disk Quotas

To determine if the FreeBSD kernel provides support for disk quotas:

```
% sysctl kern.features.ufs_quota
kern.features.ufs_quota: 1
```


In this example, the `1` indicates quota support. If the value is instead `0`, add the following line to a custom kernel configuration file and rebuild the kernel using the instructions in [Configuring the FreeBSD Kernel](#):

```
options QUOTA
```

Next, enable disk quotas in **`/etc/rc.conf`**:

```
quota_enable="YES"
```

Normally on bootup, the quota integrity of each file system is checked by [quotacheck\(8\)](#). This program insures that the data in the quota database properly reflects the data on the file system. This is a time consuming process that will significantly affect the time the system takes to boot. To skip this step, add this variable to **`/etc/rc.conf`**:

```
check_quotas="NO"
```

Finally, edit **`/etc/fstab`** to enable disk quotas on a per-file system basis. To enable per-user quotas on a file system, add `userquota` to the options field in the **`/etc/fstab`** entry for the file system to enable quotas on. For example:

```
/dev/da1s2g    /home    ufs rw,userquota 1 2
```

To enable group quotas, use `groupquota` instead. To enable both user and group quotas, separate the options with a comma:

```
/dev/da1s2g    /home    ufs rw,userquota,groupquota 1 2
```

By default, quota files are stored in the root directory of the file system as **`quota.user`** and **`quota.group`**. Refer to [fstab\(5\)](#) for more information. Specifying an alternate location for the quota files is not recommended.

Once the configuration is complete, reboot the system and **`/etc/rc`** will automatically run the appropriate commands to create the initial quota files for all of the quotas enabled in **`/etc/fstab`**.

In the normal course of operations, there should be no need to manually run [quotacheck\(8\)](#), [quotaon\(8\)](#), or [quotaoff\(8\)](#). However, one should read these manual pages to be familiar with their operation.

21.11.2. Setting Quota Limits

To verify that quotas are enabled, run:





```
# quota -v
```

There should be a one line summary of disk usage and current quota limits for each file system that quotas are enabled on.

The system is now ready to be assigned quota limits with `edquota`.

Several options are available to enforce limits on the amount of disk space a user or group may allocate, and how many files they may create. Allocations can be limited based on disk space (block quotas), number of files (inode quotas), or a combination of both. Each limit is further broken down into two categories: hard and soft limits.

A hard limit may not be exceeded. Once a user reaches a hard limit, no further allocations can be made on that file system by that user. For example, if the user has a hard limit of 500 kbytes on a file system and is currently using 490 kbytes, the user can only allocate an additional 10 kbytes. Attempting to allocate an additional 11 kbytes will fail.

Soft limits can be exceeded for a limited amount of time, known as the grace period, which is one week by default. If a user stays over their limit longer than the grace period, the soft limit turns into a hard limit and no further allocations are allowed. When the user drops back below the soft limit, the grace period is reset.

In the following example, the quota for the `test` account is being edited. When `edquota` is invoked, the editor specified by `EDITOR` is opened in order to edit the quota limits. The default editor is set to `vi`.



```
# edquota -u test
Quotas for user test:
/usr: kbytes in use: 65, limits (soft = 50, hard = 75)
      inodes in use: 7, limits (soft = 50, hard = 60)
/usr/var: kbytes in use: 0, limits (soft = 50, hard = 75)
          inodes in use: 0, limits (soft = 50, hard = 60)
```

There are normally two lines for each file system that has quotas enabled. One line represents the block limits and the other represents the inode limits. Change the value to modify the quota limit. For example, to raise the block limit on `/usr` to a soft limit of `500` and a hard limit of `600`, change the values in that line as follows:

```
/usr: kbytes in use: 65, limits (soft = 500, hard = 600)
```

The new quota limits take effect upon exiting the editor.

Sometimes it is desirable to set quota limits on a range of users. This can be done by first assigning the desired quota limit to a user. Then, use `-p` to duplicate that quota to a specified



range of user IDs (UIDs). The following command will duplicate those quota limits for UIDs 10,000 through 19,999:

```
# edquota -p test 10000-19999
```

For more information, refer to [edquota\(8\)](#).

21.11.3. Checking Quota Limits and Disk Usage

To check individual user or group quotas and disk usage, use [quota\(1\)](#). A user may only examine their own quota and the quota of a group they are a member of. Only the superuser may view all user and group quotas. To get a summary of all quotas and disk usage for file systems with quotas enabled, use [repquota\(8\)](#).

Normally, file systems that the user is not using any disk space on will not show in the output of `quota`, even if the user has a quota limit assigned for that file system. Use `-v` to display those file systems. The following is sample output from `quota -v` for a user that has quota limits on two file systems.

```
Disk quotas for user test (uid 1002):
```

Filesystem	usage	quota	limit	grace	files	quota	limit	grace
/usr	65*	50	75	5days	7	50	60	
/usr/var	0	50	75		0	50	60	

In this example, the user is currently 15 kbytes over the soft limit of 50 kbytes on **/usr** and has 5 days of grace period left. The asterisk `*` indicates that the user is currently over the quota limit.

21.11.4. Quotas over NFS

Quotas are enforced by the quota subsystem on the NFS server. The [rpc.rquotad\(8\)](#) daemon makes quota information available to `quota` on NFS clients, allowing users on those machines to see their quota statistics.

On the NFS server, enable `rpc.rquotad` by removing the `#` from this line in **/etc/inetd.conf**:

```
rquotad/1      dgram rpc/udp wait root /usr/libexec/rpc.rquotad rpc.rquotad
```

Then, restart `inetd`:

```
# service inetd restart
```

21.12. Encrypting Disk Partitions



FreeBSD offers excellent online protections against unauthorized data access. File permissions and [Mandatory Access Control](#) (MAC) help prevent unauthorized users from accessing data while the operating system is active and the computer is powered up. However, the permissions enforced by the operating system are irrelevant if an attacker has physical access to a computer and can move the computer's hard drive to another system to copy and analyze the data.

Regardless of how an attacker may have come into possession of a hard drive or powered-down computer, the GEOM-based cryptographic subsystems built into FreeBSD are able to protect the data on the computer's file systems against even highly-motivated attackers with significant resources. Unlike encryption methods that encrypt individual files, the built-in `gbde` and `geli` utilities can be used to transparently encrypt entire file systems. No cleartext ever touches the hard drive's platter.

This chapter demonstrates how to create an encrypted file system on FreeBSD. It first demonstrates the process using `gbde` and then demonstrates the same example using `geli`.

Warning

GBDE disk-encryption is deprecated, and removed in FreeBSD 15.0.

21.12.1. Disk Encryption with `gbde`

The objective of the [gbde\(4\)](#) facility is to provide a formidable challenge for an attacker to gain access to the contents of a *cold* storage device. However, if the computer is compromised while up and running and the storage device is actively attached, or the attacker has access to a valid passphrase, it offers no protection to the contents of the storage device. Thus, it is important to provide physical security while the system is running and to protect the passphrase used by the encryption mechanism.

This facility provides several barriers to protect the data stored in each disk sector. It encrypts the contents of a disk sector using 128-bit AES in CBC mode. Each sector on the disk is encrypted with a different AES key. For more information on the cryptographic design, including how the sector keys are derived from the user-supplied passphrase, refer to [gbde\(4\)](#).

FreeBSD provides a kernel module for `gbde` which can be loaded with this command:

```
# kldload geom_bde
```



If using a custom kernel configuration file, ensure it contains this line:

```
options GEOM_BDE
```

The following example demonstrates adding a new hard drive to a system that will hold a single encrypted partition that will be mounted as **/private**.



Procedure: Encrypting a Partition with gbde

1. Add the New Hard Drive

Install the new drive to the system as explained in [Adding Disks](#). For the purposes of this example, a new hard drive partition has been added as **/dev/ad4s1c** and **/dev/ad0s1*** represents the existing standard FreeBSD partitions.

```
# ls /dev/ad*  
/dev/ad0          /dev/ad0s1b      /dev/ad0s1e      /dev/ad4s1  
/dev/ad0s1        /dev/ad0s1c      /dev/ad0s1f      /dev/ad4s1c  
/dev/ad0s1a       /dev/ad0s1d      /dev/ad4
```

2. Create a Directory to Hold gbde Lock Files

```
# mkdir /etc/gbde
```

The gbde lock file contains information that gbde requires to access encrypted partitions. Without access to the lock file, gbde will not be able to decrypt the data contained in the encrypted partition without significant manual intervention which is not supported by the software. Each encrypted partition uses a separate lock file.

3. Initialize the gbde Partition

A gbde partition must be initialized before it can be used. This initialization needs to be performed only once. This command will open the default editor, in order to set various configuration options in a template. For use with the UFS file system, set the sector_size to 2048:

```
# gbde init /dev/ad4s1c -i -L /etc/gbde/ad4s1c.lock  
#  
# Sector size is the smallest unit of data which can be read or written.  
# Making it too small decreases performance and decreases available space.  
# Making it too large may prevent filesystems from working. 512 is the  
# minimum and always safe. For UFS, use the fragment size  
#  
sector_size      =      2048  
[...]
```

Once the edit is saved, the user will be asked twice to type the passphrase used to secure the data. The passphrase must be the same both times. The ability of gbde to protect data depends entirely on the quality of the passphrase. For tips on how to select a secure passphrase that is easy to remember, see

<http://world.std.com/~reinhold/diceware.htm>.



This initialization creates a lock file for the gbde partition. In this example, it is stored as **/etc/gbde/ad4s1c.lock**. Lock files must end in ".lock" in order to be correctly detected by the **/etc/rc.d/gbde** start up script.

Caution

Lock files *must* be backed up together with the contents of any encrypted partitions. Without the lock file, the legitimate owner will be unable to access the data on the encrypted partition.

4. Attach the Encrypted Partition to the Kernel

```
# gbde attach /dev/ad4s1c -l /etc/gbde/ad4s1c.lock
```

This command will prompt to input the passphrase that was selected during the initialization of the encrypted partition. The new encrypted device will appear in **/dev** as **/dev/device_name.bde**:

```
# ls /dev/ad*  
/dev/ad0          /dev/ad0s1b      /dev/ad0s1e      /dev/ad4s1  
/dev/ad0s1        /dev/ad0s1c      /dev/ad0s1f      /dev/ad4s1c  
/dev/ad0s1a       /dev/ad0s1d      /dev/ad4          /dev/ad4s1c.bde
```

5. Create a File System on the Encrypted Device

Once the encrypted device has been attached to the kernel, a file system can be created on the device. This example creates a UFS file system with soft updates enabled. Be sure to specify the partition which has a ***.bde** extension:

```
# newfs -U /dev/ad4s1c.bde
```

6. Mount the Encrypted Partition

Create a mount point and mount the encrypted file system:

```
# mkdir /private  
# mount /dev/ad4s1c.bde /private
```

7. Verify That the Encrypted File System is Available

The encrypted file system should now be visible and available for use:



```
% df -H
```

Filesystem	Size	Used	Avail	Capacity	Mounted on
/dev/ad0s1a	1037M	72M	883M	8%	/
/devfs	1.0K	1.0K	0B	100%	/dev
/dev/ad0s1f	8.1G	55K	7.5G	0%	/home
/dev/ad0s1e	1037M	1.1M	953M	0%	/tmp
/dev/ad0s1d	6.1G	1.9G	3.7G	35%	/usr
/dev/ad4s1c.bde	150G	4.1K	138G	0%	/private

After each boot, any encrypted file systems must be manually re-attached to the kernel, checked for errors, and mounted, before the file systems can be used. To configure these steps, add the following lines to **/etc/rc.conf**:

```
gbde_autoattach_all="YES"  
gbde_devices="ad4s1c"  
gbde_lockdir="/etc/gbde"
```

This requires that the passphrase be entered at the console at boot time. After typing the correct passphrase, the encrypted partition will be mounted automatically. Additional gbde boot options are available and listed in [rc.conf\(5\)](#).

i Note

sysinstall is incompatible with gbde-encrypted devices. All ***.bde** devices must be detached from the kernel before starting sysinstall or it will crash during its initial probing for devices. To detach the encrypted device used in the example, use the following command:

```
# gbde detach /dev/ad4s1c
```

21.12.2. Disk Encryption with geli

An alternative cryptographic GEOM class is available using **geli**. This control utility adds some features and uses a different scheme for doing cryptographic work. It provides the following features:

- Utilizes the [crypto\(9\)](#) framework and automatically uses cryptographic hardware when it is available.
- Supports multiple cryptographic algorithms such as AES-XTS, AES-CBC, and Camellia-CBC



- Allows the root partition to be encrypted. The passphrase used to access the encrypted root partition will be requested during system boot.
- Allows the use of two independent keys.
- It is fast as it performs simple sector-to-sector encryption.
- Allows backup and restore of master keys. If a user destroys their keys, it is still possible to get access to the data by restoring keys from the backup.
- Allows a disk to attach with a random, one-time key which is useful for swap partitions and temporary file systems.

More features and usage examples can be found in [geli\(8\)](#).

The following example describes how to generate a key file which will be used as part of the master key for the encrypted provider mounted under **/private**. The key file will provide some random data used to encrypt the master key. The master key will also be protected by a passphrase. The provider's sector size will be 4kB. The example describes how to attach to the `geli` provider, create a file system on it, mount it, work with it, and finally, how to detach it.

Procedure: Encrypting a Partition with `geli`

1. Load `geli` Support

Support for `geli` is available as a loadable kernel module. To configure the system to automatically load the module at boot time, add the following line to **/boot/loader.conf**:

```
geom_eli_load="YES"
```

To load the kernel module now:

```
# kldload geom_eli
```

For a custom kernel, ensure the kernel configuration file contains these lines:

```
options GEOM_ELI
device crypto
```

2. Generate the Master Key

The following commands generate a master key that all data will be encrypted with. This key can never be changed. Rather than using it directly, it is encrypted with one or more user keys. The user keys are made up of an optional combination of random bytes from a file, **/root/da2.key**, and/or a passphrase. In this case, the data source for the key file is



/dev/random. This command also configures the sector size of the provider (**/dev/da2.eli**) as 4kB, for better performance:

```
# dd if=/dev/random of=/root/da2.key bs=64 count=1
# geli init -K /root/da2.key -s 4096 /dev/da2
Enter new passphrase:
Reenter new passphrase:
```

It is not mandatory to use both a passphrase and a key file as either method of securing the master key can be used in isolation.

If the key file is given as "-", standard input will be used. For example, this command generates three key files:

```
# cat keyfile1 keyfile2 keyfile3 | geli init -K - /dev/da2
```

3. Attach the Provider with the Generated Key

To attach the provider, specify the key file, the name of the disk, and the passphrase:

```
# geli attach -k /root/da2.key /dev/da2
Enter passphrase:
```

This creates a new device with an **.eli** extension:

```
# ls /dev/da2*
/dev/da2  /dev/da2.eli
```

4. Create the New File System

Next, format the device with the UFS file system and mount it on an existing mount point:

```
# dd if=/dev/random of=/dev/da2.eli bs=1m
# newfs /dev/da2.eli
# mount /dev/da2.eli /private
```

The encrypted file system should now be available for use:

```
# df -H
Filesystem      Size  Used Avail Capacity  Mounted on
```

/dev/ad0s1a	248M	89M	139M	38%	/
/devfs	1.0K	1.0K	0B	100%	/dev
/dev/ad0s1f	7.7G	2.3G	4.9G	32%	/usr
/dev/ad0s1d	989M	1.5M	909M	0%	/tmp
/dev/ad0s1e	3.9G	1.3G	2.3G	35%	/var
/dev/da2.eli	150G	4.1K	138G	0%	/private

Once the work on the encrypted partition is done, and the **/private** partition is no longer needed, it is prudent to put the device into cold storage by unmounting and detaching the **geli** encrypted partition from the kernel:

```
# umount /private
# geli detach da2.eli
```

An **rc.d** script is provided to simplify the mounting of **geli**-encrypted devices at boot time. For this example, add these lines to **/etc/rc.conf**:

```
geli_devices="da2"
geli_da2_flags="-k /root/da2.key"
```

This configures **/dev/da2** as a **geli** provider with a master key of **/root/da2.key**. The system will automatically detach the provider from the kernel before the system shuts down. During the startup process, the script will prompt for the passphrase before attaching the provider. Other kernel messages might be shown before and after the password prompt. If the boot process seems to stall, look carefully for the password prompt among the other messages. Once the correct passphrase is entered, the provider is attached. The file system is then mounted, typically by an entry in **/etc/fstab**. Refer to “[Mounting and Unmounting File Systems](#)” for instructions on how to configure a file system to mount at boot time.

21.13. Encrypting Swap

Like the encryption of disk partitions, encryption of swap space is used to protect sensitive information. Consider an application that deals with passwords. As long as these passwords stay in physical memory, they are not written to disk and will be cleared after a reboot. However, if FreeBSD starts swapping out memory pages to free space, the passwords may be written to the disk unencrypted. Encrypting swap space can be a solution for this scenario.

This section demonstrates how to configure an encrypted swap partition using [gbde\(8\)](#) or [geli\(8\)](#) encryption. It assumes that **/dev/ada0s1b** is the swap partition.

21.13.1. Configuring Encrypted Swap



Swap partitions are not encrypted by default and should be cleared of any sensitive data before continuing. To overwrite the current swap partition with random garbage, execute the following command:

```
# dd if=/dev/random of=/dev/ada0s1b bs=1m
```

To encrypt the swap partition using [gbde\(8\)](#), add the `.bde` suffix to the swap line in **/etc/fstab**:

# Device	Mountpoint	FStype	Options	Dump	Pass#
/dev/ada0s1b.bde	none	swap	sw	0	0

To instead encrypt the swap partition using [geli\(8\)](#), initialize the device with a onetime key:

```
# geom eli onetime -d /dev/ada0s1b
```

Afterwards, add a line for the device with the `.eli` suffix to **/etc/fstab**:

# Device	Mountpoint	FStype	Options	Dump	Pass#
/dev/ada0s1b.eli	none	swap	sw	0	0

By default, [geli\(8\)](#) uses the AES algorithm with a key length of 128 bits. Normally the default settings will suffice. If desired, these defaults can be altered in the options field in **/etc/fstab**. The possible flags are:

aalgo

Data integrity verification algorithm used to ensure that the encrypted data has not been tampered with. See [geli\(8\)](#) for a list of supported algorithms.

ealgo

Encryption algorithm used to protect the data. See [geli\(8\)](#) for a list of supported algorithms.

keylen

The length of the key used for the encryption algorithm. See [geli\(8\)](#) for the key lengths that are supported by each encryption algorithm.

sectorsize

The size of the blocks data is broken into before it is encrypted. Larger sector sizes increase performance at the cost of higher storage overhead. The recommended size is 4096 bytes.

This example configures an encrypted swap partition using the AES-XTS algorithm with a key length of 128 bits and a sectorsize of 4 kilobytes:



# Device	Mountpoint	FStype	Options	Dum
/dev/ada0s1b.eli	none	swap	sw,ealgo=AES-XTS,keylen=128,sectors	

21.13.2. Encrypted Swap Verification

Once the system has rebooted, proper operation of the encrypted swap can be verified using `swapinfo`.

If `gbde(8)` is being used:

```
% swapinfo
Device          1K-blocks      Used      Avail Capacity
/dev/ada0s1b.bde  542720          0    542720       0
```

If `geli(8)` is being used:

```
% swapinfo
Device          1K-blocks      Used      Avail Capacity
/dev/ada0s1b.eli  542720          0    542720       0
```

21.14. Highly Available Storage (HAST)

High availability is one of the main requirements in serious business applications and highly-available storage is a key component in such environments. In FreeBSD, the Highly Available Storage (HAST) framework allows transparent storage of the same data across several physically separated machines connected by a TCP/IP network. HAST can be understood as a network-based RAID1 (mirror), and is similar to the DRBD® storage system used in the GNU/Linux® platform. In combination with other high-availability features of FreeBSD like CARP, HAST makes it possible to build a highly-available storage cluster that is resistant to hardware failures.

The following are the main features of HAST:

- Can be used to mask I/O errors on local hard drives.
- File system agnostic as it works with any file system supported by FreeBSD.
- Efficient and quick resynchronization as only the blocks that were modified during the downtime of a node are synchronized.
- Can be used in an already deployed environment to add additional redundancy.
- Together with CARP, Heartbeat, or other tools, it can be used to build a robust and durable storage system.

Read this chapter to learn:



- What HAST is, how it works, and which features it provides.
- How to set up and use HAST on FreeBSD.
- How to integrate CARP and [devd\(8\)](#) to build a robust storage system.

Before reading this section:

- Understand UNIX® and FreeBSD basics ([FreeBSD Basics](#)).
- Know how to configure network interfaces and other core FreeBSD subsystems ([Configuration and Tuning](#)).
- Have a good understanding of FreeBSD networking ([Network Communication](#)).

The HAST project was sponsored by The FreeBSD Foundation with support from <http://www.omc.net/> and <http://www.transip.nl/>.

21.14.1. HAST Operation

HAST provides synchronous block-level replication between two physical machines: the *primary* node and the *secondary* node. These two machines together are referred to as a cluster.

Since HAST works in a primary-secondary configuration, it allows only one of the cluster nodes to be active at any given time. The primary node, also called *active*, is the one which will handle all the I/O requests to HAST-managed devices. The secondary node is automatically synchronized from the primary node.

The physical components of the HAST system are the local disk on primary node, and the disk on the remote, secondary node.

HAST operates synchronously on a block level, making it transparent to file systems and applications. HAST provides regular GEOM providers in **/dev/hast/** for use by other tools or applications. There is no difference between using HAST-provided devices and raw disks or partitions.

Each write, delete, or flush operation is sent to both the local disk and to the remote disk over TCP/IP. Each read operation is served from the local disk, unless the local disk is not up-to-date or an I/O error occurs. In such cases, the read operation is sent to the secondary node.

HAST tries to provide fast failure recovery. For this reason, it is important to reduce synchronization time after a node's outage. To provide fast synchronization, HAST manages an on-disk bitmap of dirty extents and only synchronizes those during a regular synchronization, with an exception of the initial sync.

There are many ways to handle synchronization. HAST implements several replication modes to handle different synchronization methods:

- *memsync*: This mode reports a write operation as completed when the local write operation is finished and when the remote node acknowledges data arrival, but before actually storing the data. The data on the remote node will be stored directly after sending the



acknowledgement. This mode is intended to reduce latency, but still provides good reliability. This mode is the default.

- *fullsync*: This mode reports a write operation as completed when both the local write and the remote write complete. This is the safest and the slowest replication mode.
- *async*: This mode reports a write operation as completed when the local write completes. This is the fastest and the most dangerous replication mode. It should only be used when replicating to a distant node where latency is too high for other modes.

21.14.2. HAST Configuration

The HAST framework consists of several components:

- The [hastd\(8\)](#) daemon which provides data synchronization. When this daemon is started, it will automatically load `geom_gate.ko`.
- The userland management utility, [hastctl\(8\)](#).
- The [hast.conf\(5\)](#) configuration file. This file must exist before starting `hastd`.

Users who prefer to statically build `GEOM_GATE` support into the kernel should add this line to the custom kernel configuration file, then rebuild the kernel using the instructions in [Configuring the FreeBSD Kernel](#):

```
options GEOM_GATE
```

The following example describes how to configure two nodes in primary-secondary operation using HAST to replicate the data between the two. The nodes will be called `hasta`, with an IP address of `172.16.0.1`, and `hastb`, with an IP address of `172.16.0.2`. Both nodes will have a dedicated hard drive `/dev/ad6` of the same size for HAST operation. The HAST pool, sometimes referred to as a resource or the GEOM provider in `/dev/hast/`, will be called `test`.

Configuration of HAST is done using `/etc/hast.conf`. This file should be identical on both nodes. The simplest configuration is:

```
resource test {
    on hasta {
        local /dev/ad6
        remote 172.16.0.2
    }
    on hastb {
        local /dev/ad6
        remote 172.16.0.1
    }
}
```



For more advanced configuration, refer to [hast.conf\(5\)](#).

Tip

It is also possible to use host names in the `remote` statements if the hosts are resolvable and defined either in `/etc/hosts` or in the local DNS.

Once the configuration exists on both nodes, the HAST pool can be created. Run these commands on both nodes to place the initial metadata onto the local disk and to start [hastd\(8\)](#):

```
# hastctl create test
# service hastd onestart
```



Note

It is *not* possible to use GEOM providers with an existing file system or to convert an existing storage to a HAST-managed pool. This procedure needs to store some metadata on the provider and there will not be enough required space available on an existing provider.

A HAST node's `primary` or `secondary` role is selected by an administrator, or software like Heartbeat, using [hastctl\(8\)](#). On the primary node, `hastctl`, issue this command:

```
# hastctl role primary test
```



Run this command on the secondary node, `hastctl`:

```
# hastctl role secondary test
```



Verify the result by running `hastctl` on each node:

```
# hastctl status test
```



Check the `status` line in the output. If it says `degraded`, something is wrong with the configuration file. It should say `complete` on each node, meaning that the synchronization between the nodes has started. The synchronization completes when `hastctl status` reports 0 bytes of `dirty` extents.

The next step is to create a file system on the GEOM provider and mount it. This must be done on the `primary` node. Creating the file system can take a few minutes, depending on the size of the



hard drive. This example creates a UFS file system on **/dev/hast/test**:

```
# newfs -U /dev/hast/test
# mkdir /hast/test
# mount /dev/hast/test /hast/test
```

Once the HAST framework is configured properly, the final step is to make sure that HAST is started automatically during system boot. Add this line to **/etc/rc.conf**:

```
hastd_enable="YES"
```

21.14.2.1. Failover Configuration

The goal of this example is to build a robust storage system which is resistant to the failure of any given node. If the primary node fails, the secondary node is there to take over seamlessly, check and mount the file system, and continue to work without missing a single bit of data.

To accomplish this task, the Common Address Redundancy Protocol (CARP) is used to provide for automatic failover at the IP layer. CARP allows multiple hosts on the same network segment to share an IP address. Set up CARP on both nodes of the cluster according to the documentation available in [“Common Address Redundancy Protocol \(CARP\)”](#). In this example, each node will have its own management IP address and a shared IP address of *172.16.0.254*. The primary HAST node of the cluster must be the primary CARP node.

The HAST pool created in the previous section is now ready to be exported to the other hosts on the network. This can be accomplished by exporting it through NFS or Samba, using the shared IP address *172.16.0.254*. The only problem which remains unresolved is an automatic failover should the primary node fail.

In the event of CARP interfaces going up or down, the FreeBSD operating system generates a [devd\(8\)](#) event, making it possible to watch for state changes on the CARP interfaces. A state change on the CARP interface is an indication that one of the nodes failed or came back online. These state change events make it possible to run a script which will automatically handle the HAST failover.

To catch state changes on the CARP interfaces, add this configuration to **/etc/devd.conf** on each node, while replacing `<vhid>` with the virtual host id and `<ifname>` with the associated interface name:

```
notify 30 {
    match "system" "CARP";
    match "subsystem" "<vhid>@<ifname>";
    match "type" "MASTER";
    action "/usr/local/sbin/carp-hast-switch primary";
};
```



```

notify 30 {
    match "system" "CARP";
    match "subsystem" "<vhid>@<ifname>";
    match "type" "BACKUP";
    action "/usr/local/sbin/carp-hast-switch secondary";
};

```

Restart [devd\(8\)](#) on both nodes to put the new configuration into effect:

```
# service devd restart
```

When the specified interface state changes by going up or down , the system generates a notification, allowing the [devd\(8\)](#) subsystem to run the specified automatic failover script, **/usr/local/sbin/carp-hast-switch**. For further clarification about this configuration, refer to [devd.conf\(5\)](#).

Here is an example of an automated failover script:

```

#!/bin/sh

# Original script by Freddie Cash <fjwcash@gmail.com>
# Modified by Michael W. Lucas <mwllucas@BlackHelicopters.org>
# and Viktor Petersson <vpetersson@wireload.net>

# The names of the HAST resources, as listed in /etc/hast.conf
resources="test"

# delay in mounting HAST resource after becoming primary
# make your best guess
delay=3

# logging
log="local0.debug"
name="carp-hast"

# end of user configurable stuff

case "$1" in
    primary)
        logger -p $log -t $name "Switching to primary provider for ${resources}"
        sleep ${delay}

        # Wait for any "hastd secondary" processes to stop
        for disk in ${resources}; do
            while $( pgrep -lf "hastd: ${disk} \ (secondary\)" > /dev/null )
            do
                sleep 1
            done
        done
    esac

```

```

        # Switch role for each disk
        hastctl role primary ${disk}
        if [ $? -ne 0 ]; then
            logger -p $log -t $name "Unable to change role to p
            exit 1
        fi
    done

    # Wait for the /dev/hast/* devices to appear
    for disk in ${resources}; do
        for I in $( jot 60 ); do
            [ -c "/dev/hast/${disk}" ] && break
            sleep 0.5
        done

        if [ ! -c "/dev/hast/${disk}" ]; then
            logger -p $log -t $name "GEOM provider /dev/hast/${
            exit 1
        fi
    done

    logger -p $log -t $name "Role for HAST resources ${resources} switc

    logger -p $log -t $name "Mounting disks."
    for disk in ${resources}; do
        mkdir -p /hast/${disk}
        fsck -p -y -t ufs /dev/hast/${disk}
        mount /dev/hast/${disk} /hast/${disk}
    done

;;

secondary)
    logger -p $log -t $name "Switching to secondary provider for ${reso

    # Switch roles for the HAST resources
    for disk in ${resources}; do
        if ! mount | grep -q "^/dev/hast/${disk} on "
        then
        else
            umount -f /hast/${disk}
        fi
        sleep $delay
        hastctl role secondary ${disk} 2>&1
        if [ $? -ne 0 ]; then
            logger -p $log -t $name "Unable to switch role to s
            exit 1
        fi
        logger -p $log -t $name "Role switched to secondary for res

```



```
done
;;
esac
```

In a nutshell, the script takes these actions when a node becomes primary:

- Promotes the HAST pool to primary on the other node.
- Checks the file system under the HAST pool.
- Mounts the pool.

When a node becomes secondary:

- Unmounts the HAST pool.
- Degrades the HAST pool to secondary.

Caution

This is just an example script which serves as a proof of concept. It does not handle all the possible scenarios and can be extended or altered in any way, for example, to start or stop required services.

Tip

For this example, a standard UFS file system was used. To reduce the time needed for recovery, a journal-enabled UFS or ZFS file system can be used instead.

Instead of using the highly available storage locally, it can also be shared to other computers on a network via [NFS](#), [iSCSI](#), [sshfs\(1\)](#), or programs in ports (i.e. [net/samba419](#)).

More detailed information with additional examples can be found at <http://wiki.FreeBSD.org/HAST>.

21.14.3. Troubleshooting

HAST should generally work without issues. However, as with any other software product, there may be times when it does not work as supposed. The sources of the problems may be different, but the rule of thumb is to ensure that the time is synchronized between the nodes of the cluster.

When troubleshooting HAST, the debugging level of [hastd\(8\)](#) should be increased by starting `hastd` with `-d`. This argument may be specified multiple times to further increase the debugging level. Consider also using `-F`, which starts `hastd` in the foreground.

21.14.3.1. Recovering from the Split-brain Condition



Split-brain occurs when the nodes of the cluster are unable to communicate with each other, and both are configured as primary. This is a dangerous condition because it allows both nodes to make incompatible changes to the data. This problem must be corrected manually by the system administrator.

The administrator must either decide which node has more important changes, or perform the merge manually. Then, let HAST perform full synchronization of the node which has the broken data. To do this, issue these commands on the node which needs to be resynchronized:

hastctl role init test
hastctl create test
hastctl role secondary test

Last modified on: June 27, 2026 by [Benedict Reuschling](#)



 English

System 

About

- FreeBSD
- FreeBSD Foundation
- Get FreeBSD
- Code of Conduct
- Security Advisories

Documentation

- Documentation portal
- Manual pages
- Presentations and papers
- Previous versions



4.4BSD Documents

Wiki

Community

Get involved

Community forum

Mailing lists

IRC Channels

Bug Tracker

Legal

Donations

Licensing

Privacy Policy

Legal notices

© 1994-2026 The FreeBSD Project. All rights reserved

Made with  by the FreeBSD Community

